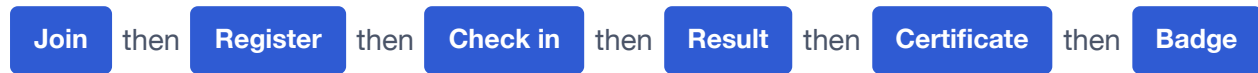


Campus Clubs

One place for SRM clubs to find members, run events, take attendance, share results and give certificates and badges.



Version	1.0
Date	3 October 2026
Built by	One student developer (frontend, backend and deploy)
For	All SRM campuses · @srmist.edu.in
Status	Phase 1 backend done (27 tests pass). Frontend not started.

What is inside

1. The problem
2. Why I am solving it
3. Who will use it
4. What it must do
5. How I will build it (my approach)
6. The tech stack
7. How the parts fit together
8. Keeping it safe
9. Handling many people at once
10. My three jobs: frontend, backend, deploy
11. The phases and where each one goes live
12. How every release ships
13. Risks and plans
14. Where I learn from
15. Word list

1. The problem

SRM has many clubs, and most of them sit under the engineering departments. Every club runs on the same tools: Google Forms and WhatsApp groups. These tools do not talk to each other, so a lot gets lost.

A story from today

Riya is in second year. She wants to join the Robotics Club.

1. She sees a Google Form link in a WhatsApp group. She fills it in. **She never hears back.**
2. A month later, a friend tells her about a robotics contest. She was not in the right WhatsApp group, so **she almost missed it.**
3. On the day, the room changes. The message gets lost in 200 other messages. **She goes to the wrong room.**
4. She takes part and her team comes second. **The result is never posted anywhere.**
5. Later, a company asks for proof. **She has nothing to show.**

What goes wrong, step by step

Step	Tool used today	What goes wrong
Finding new members	Google Form	Every club makes its own form with its own rules. Only the leader sees the answers. People who apply never hear back.
Telling people about an event	WhatsApp group	Messages get buried. People outside the group never know. Room or time changes get missed.
Signing up for an event	Another Google Form	No seat limit. Same person signs up twice. No way to check students from other colleges.
Attendance	Paper or a sheet	Slow and easy to fake. Often never saved.
Results	Nowhere	Winners are said out loud on stage and then forgotten.
Proof you took part	Hand-made certificate, if any	Easy to fake. Nobody can check if it is real.

The root problem: each step lives in a different tool, so nothing flows from one step to the next. The data breaks between *join*, *sign up*, *attend*, *result* and *proof*.

2. Why I am solving it

- **I live this problem.** I am an SRM student. I see these forms and WhatsApp groups every week, and I know how the clubs really work.
- **It helps many people.** Thousands of students, and every club leader who loses a whole weekend to forms and sheets after each event.
- **It gives students real proof.** A profile with results, certificates and badges that anyone can check.
- **I want to learn full-stack engineering for real.** Frontend, backend, database, security and deploying, all on a product people actually use.
- **It has been done before, and it worked.** A team built a similar project called CampusConnect and won a Best Project award. I studied their code. I will keep what worked and fix the weak spots I found.

What I learned from CampusConnect

Keep (it worked well)	Fix (weak spots I found in their code)
• Clean layers: routes, then logic, then database • Club leader is a <i>membership</i>, not a login type • Certificates with a QR code that links to a public check page • The AI only phrases answers. Code picks the clubs, so the AI cannot make them up.	• Anyone could sign up as an admin. Here, only I can make an owner, from the command line. • Anyone could type a fake college email. Here, SRM users must sign in with their real Google account. • Login tokens lasted 28 days in browser storage. Here, sessions sit in a safe cookie that page code can't read. • Certificate numbers could be guessed. Here, they are long random codes. • Attendance was ticked by hand. Here, it is a QR scan.

3. Who will use it

There is **no "campus admin"**. Each club runs itself. I (the developer) only decide which clubs get in.

Role	Who	What they can do
Platform Owner	Me, the developer	Say yes or no to new clubs. Pause a club. Give a club to a new leader. Set a few rules for everyone, like the most clubs one student can join. Cannot see inside clubs.
Club Admin	Club president or head	Runs everything for their own club: hiring rules, the hiring queue, events, approving outside guests, results, certificates and posts. Can make other members Core or Admin.
Core Member	Club core team	Makes events, scans people in at the door, posts event updates. Cannot change club rules or roles.
Member	Anyone picked by the club	Sees member-only events and posts.
SRM Student	Anyone with <code>@srmist.edu.in</code>	Signs in with Google. Applies to clubs. Signs up for events and is confirmed right away. Earns certificates and badges.
Outside Guest	Student from another college	Signs in with a personal email and a 6-digit code. Can sign up for open events, but must wait for the event team to say yes. Cannot join clubs.

4. What it must do

A. Hiring club members (replaces Google Forms)

- The Club Admin sets the rules **once**: open and close dates, who can apply (year, department), how many seats, and the questions.
- Every application goes into **one queue**.
- Each person moves through steps: **Applied, Shortlisted, Interview, Selected** or **Not selected**.
- The student sees their step live and gets told when it changes.
- Picking someone as "Selected" makes them a member right away.

B. Events (replaces WhatsApp)

- Each event has its own page: room, time, map link, seats left and live status.
- Changes like "Room moved to Hall B" or "Starting now" reach everyone signed up by email and phone notification.
- SRM students are confirmed right away. Outside guests wait for a yes from the event team.
- The app never lets more people in than there are seats.

C. One flow: sign up, check in, result, certificate, badge

- Check-in is a QR scan at the door. The code changes every 30 seconds, so a screenshot shared on WhatsApp will not work.
- Results go on the event page and stay there forever.
- Certificates are made by the app, with a QR code anyone can scan to check they are real.

D. One club list, one profile, real proof

- A list of every club. You can filter it by campus, department and type. Your own campus shows first.
- Every student gets a public page at `/u/their-name` with clubs, events, results, certificates and badges.

E. Badges, like GitHub

- The app gives badges by itself, based on what really happened. Examples: *First Event, Hackathon x3, Winner, Core Member*.
- Badges have levels like GitHub's (x2, x3, x4).
- Each badge has a public check link.
- A small badge card can be added to a GitHub README or a LinkedIn profile.

Not in version 1: paid events, chat between students, a phone app from the app store (the website works well on phones) and AI features. The AI club finder comes after the main flow works.

How we know it works

Goal	Target
Clubs hire without Google Forms	3 pilot clubs run a full hiring drive in the app
Leaders save time	From "event ended" to "certificates sent" in under 10 minutes
Results are never lost	Every event in the app has public results
Students use their proof	Students share their profile link or badge card
It stays up during a rush	3,000 sign-ups in 60 seconds with no errors and no overbooking

5. How I will build it (my approach)

1. **Start from the problem, not the code.** Every feature must fix a step in the "what goes wrong" table.
2. **Build thin slices.** Each phase works from start to end: backend, frontend, tests and a live link. No half-built parts sitting around.
3. **Ship every phase.** Every phase ends with something live that people can click.
4. **Safe by default.** The database itself blocks bad data, like a winner who never checked in. Every permission goes through one check function.
5. **Test the rules.** Each important rule gets a test. Tests run by themselves on every change.
6. **Real users early.** Start with 3 clubs at one campus (KTR), learn, then open to all.
7. **Write it down.** The docs, the database file and this PRD live in the GitHub repo and are updated with each phase.

6. The tech stack

I chose these tools for three reasons: they are used by real companies (good to learn), they are safe by default, and they can grow without a rewrite.

Part	Tool	Why, in simple words
Website (frontend)	Next.js (React + TypeScript)	The most used way to build React sites. Pages like the club list load fast and show up on Google.
Look and feel	Tailwind CSS + shadcn/ui	Ready-made, clean parts (buttons, forms, tables) that work for everyone, including screen readers.
Forms and data	React Hook Form, Zod, TanStack Query	Checks forms before sending. Keeps data fresh and retries when the network is slow.
QR scanner	Browser camera	Core members scan in their phone browser. Nothing to install.
Server (backend)	FastAPI (Python)	Fast. Checks every input. Makes API docs by itself. Python has the best tools for PDFs, QR codes and AI.
Database	PostgreSQL	Strong rules, so bad data can't get in. Handles many people at once.
Database code	SQLAlchemy 2 + Alembic	Talks to the database from Python. Alembic keeps track of database changes.
Fast memory	Redis	Holds logins, counts tries (to stop spam) and holds the job line.
Background jobs	arq (on Redis)	Slow work like 500 certificates or emails happens in the back, so pages stay fast.
Files	Cloudflare R2	Stores certificates and pictures. No fee each time someone downloads.
Email	Resend or Amazon SES	Sends sign-in codes and updates. Cheap and reliable.
Tests	pytest, Vitest, Playwright	Check the server, the website parts, and full clicks in a real browser.
Auto checks	GitHub Actions	Runs all tests on every push.
Watching for errors	Sentry	Tells me when something breaks for a user, and where.

Where each part lives

Part	Home	Why
Project page and docs	GitHub Pages	Free. Good for a simple page that never changes by itself. It can't run a server, so it holds only the project page, this PRD and the API docs.
The web app	Vercel	Made for Next.js. Every pull request gets its own preview link.
The API and job worker	Render (or Railway)	Runs a Python server all the time. Vercel is not made for always-on workers.
Database	Neon (PostgreSQL)	Free to start, with backups.
Redis	Upstash	Free to start, pay only for use.

7. How the parts fit together

People

Students, club teams, outside guests and me, on a laptop or phone browser

Web app on Vercel (Next.js)

Pages, forms, QR scanner. Talks to the API with a safe login cookie.

API on Render (FastAPI)

Checks who you are. Checks what you are allowed to do. Follows the rules. Writes the audit log.

PostgreSQL (Neon)

Users, clubs, hiring, events, results, certificates, badges

Redis (Upstash)

Logins, spam limits, the job line

Job worker

Makes certificates, sends emails and notifications, gives badges

R2 storage

Certificate PDFs, logos, pictures

Google sign-in

Proves a user really owns an @srmist.edu.in account

Email service

Sign-in codes, updates

The code inside the API

Folder	What it holds
app/routers/	The web addresses (endpoints). Short. They call the checks and the rules.
app/permissions.py	One <code>can(user, action, club)</code> check used by every endpoint.
app/security/	Sessions, Google check, email codes, spam limits.
app/models.py	The database tables as Python classes.
db/schema.sql	The full database. The single source of truth.
tests/	One test for each important rule.

8. Keeping it safe

Danger	What we do
Someone pretends to be an SRM student	SRM users sign in only with Google. The server checks the account belongs to <code>srmist.edu.in</code> . A personal Gmail is turned away.
Someone makes themselves an admin	No sign-up form can make an owner. Only I can, from the command line. Club Admins come only from approved clubs.
Someone steals a login	The login is a random code in a locked cookie that page code can't read. You can log out of all devices.
A bad site tricks your browser into a click	Every change needs a secret token that only our own website knows (CSRF protection).
Someone guesses email codes	Codes expire in 10 minutes, die after 5 wrong tries, and requests are limited.
A friend shares a check-in QR on WhatsApp	The QR code changes every 30 seconds and works only once.
Someone fakes a certificate	Every certificate has a long random code and a public check page.
A leader does something wrong	Every admin, core and owner action is written to an audit log: who, what, before, after and when.
Bad data gets in	Every input is checked. The database also blocks bad data on its own, like signing up twice or a winner who never came.
Private data leaks	Collect only what we need. Students can delete their account. This follows India's DPDP Act 2023.

9. Handling many people at once

The hard moment is not "many users in total". It is **a rush**: a hackathon opens 300 seats, and 3,000 people click in the same minute.

1. **No overbooking.** A seat is taken in one database step that only works if a seat is left. People can't sign up twice.
2. **Many copies of the server.** The server keeps nothing in its own memory. Logins sit in Redis, so 1 copy or 20 copies all work the same.
3. **Slow work goes to the job line.** The leader clicks "Publish results" and gets an answer at once. Certificates and emails come a minute later.
4. **Public pages are saved near users (CDN).** The club list, results and profiles load fast and barely touch the server.
5. **Database care.** Indexes for common searches, page-by-page lists, and a pool of open connections.
6. **Practice the rush.** Before the first big event, a test tool (k6) fakes 3,000 sign-ups in 60 seconds.

10. My three jobs: frontend, backend, deploy

I do all three jobs myself. Writing them down keeps each job clear, and shows what to hand over if more people join later.

Frontend Engineer: "what people see and click"

- Builds every page in Next.js: sign in, club list, club page, hiring form, queue, events, scanner, profile.
- Makes it work well on phones first, since most students use phones.
- Makes it easy for everyone to use (screen readers, keyboard, clear colors).
- Shows clear messages when something goes wrong, and loading states while waiting.
- Uses the API types made from the backend, so the two never disagree.
- Writes Vitest tests for parts and Playwright tests for full click paths.

Backend Engineer: "the rules and the data"

- Designs the database and keeps `db/schema.sql` up to date.
- Builds the API endpoints in FastAPI.
- Owns sign-in, sessions and the `can()` permission check.
- Builds the background jobs: certificates, emails, badges.
- Writes a test for every rule (seat limits, who can do what, and so on).
- Keeps the API docs correct.

Developer / DevOps Engineer: "getting it live and keeping it up"

- Sets up GitHub: branches, pull requests, issue board, GitHub Actions.
- Deploys the web app to Vercel, the API to Render, and sets up Neon, Upstash and R2.
- Keeps secrets (keys and passwords) out of the code, in each host's secret settings.
- Sets up Sentry for errors, daily database backups and uptime checks.
- Runs the rush test before big events.
- Publishes the project page and docs on GitHub Pages.

Who does what in each phase

Phase	Frontend	Backend	Developer / DevOps
0	Pick colors and fonts; simple landing page	Done: schema and plan	Repo, CI, GitHub Pages, Vercel and Render accounts
1	Sign in, setup, club list, club request, hiring form and queue	Done: sign-in, clubs, roles, hiring (27 tests)	First Vercel and Render deploy, Neon, Upstash, Google client ID

2	Event pages, sign-up, guest approval queue, live updates	Events, seats, guest approval, updates, notifications	Email service, push keys
3	QR scanner, results page, certificate check page	Check-in codes, results, certificate PDF jobs	R2 storage, job worker, rush test
4	Profiles, badge wall, badge card	Badge rules engine, profile API	CDN cache for public pages
5	Fixes from pilot feedback	Fixes from pilot feedback	Monitoring, backups, launch checklist
6	AI club finder chat	Grounded AI (Claude API)	Cost limits on AI

11. The phases and where each one goes live

Each phase ends with a live link. **GitHub Pages** holds the project page and docs. **Vercel** holds the real web app. Every pull request also gets its own Vercel preview link to test before merging. Times are for one student working part-time.

Phase 0: Set up

about 1 week

We build	GitHub repo, folders, auto tests, project landing page, this PRD.
Goes live on	GitHub Pages: project page, PRD and docs.
Done when	The landing page is live, and tests run on every push.

Phase 1: Accounts, clubs and hiring

backend done

We build	Google sign-in for SRM, email codes for guests, first-time setup (campus, department, year), club requests, my review queue, club roles, hiring drives with forms, the application queue and its steps.
Goes live on	Vercel: first version of the web app. Render: the API.
Done when	A real student signs in with SRM Google, requests a club, I approve it, and the club hires one member in the app.

Phase 2: Events

about 2–3 weeks

We build	Make and publish events, sign up with seat limits, guest approval queue, live updates ("room moved"), email and phone notifications.
Goes live on	Vercel update.
Done when	A club runs a real event sign-up with no Google Form and no WhatsApp message needed.

Phase 3: Check-in, results and certificates

about 2–3 weeks

We build	QR check-in with changing codes, results page, certificate PDFs made in the background, a public certificate check page.
Goes live on	Vercel update. The rush test (3,000 sign-ups in 60 seconds) is run.
Done when	A leader goes from "event ended" to "certificates sent" in under 10 minutes.

Phase 4: Badges and profiles

about 2 weeks

We build	Public profile pages, badge rules, badge levels (x2, x3, x4), badge check links, the badge card for GitHub and LinkedIn.
Goes live on	Vercel update. Public pages are cached on the CDN.
Done when	A student adds their badge card to their GitHub README.

Phase 5: Pilot and launch

about 3–4 weeks

We build	Fixes from real use. Help pages. Error alerts, backups and uptime checks.
Goes live on	Vercel on the real web address. Pilot with 3 clubs at KTR, then open to all SRM campuses.
Done when	The 3 pilot clubs say they no longer need Google Forms.

Phase 6: AI club finder (later)

about 2 weeks

We build	"Tell me what you like" chat that suggests clubs and events. The code picks the clubs, and the AI only writes the reply, so it can't make clubs up.
Goes live on	Vercel update.
Done when	Students find a club through the chat, and costs stay inside a set limit.

12. How every release ships

1. **Small branch.** One feature per branch, for example `feat/event-signup`.
2. **Pull request.** Explain what changed and why, in plain words.
3. **Auto tests.** GitHub Actions runs every backend and frontend test. Red means stop.
4. **Preview link.** Vercel makes a preview link. I click through the change on a phone and a laptop.
5. **Merge.** Merging to `main` deploys the web app (Vercel) and the API (Render) by itself.
6. **Database changes.** Alembic runs the change before the new API starts. Take a backup first.
7. **Check.** Look at Sentry and the health check for 15 minutes after each release.
8. **Update the docs.** Update the README status table and this PRD on GitHub Pages.

Before each phase goes live

Check
All tests pass.
No secrets in the code.
New endpoints use the <code>can()</code> check and write to the audit log.
Pages work on a small phone screen.
A database backup was taken today.
Rollback plan: redeploy the last good version from Vercel and Render with one click.

13. Risks and plans

Risk	Plan
Clubs keep using WhatsApp out of habit	Start with 3 friendly clubs. Make the app faster than a form. Let leaders share event links into WhatsApp.
One person (me) holds the owner role	More owners can be added from the command line, so the app can be handed over after I graduate.
Free hosting limits	Watch usage. Move to paid plans only when real use needs it.
Wrong campus or department list	Check the list with the college before the pilot. It is easy to change.
Big rush breaks sign-up	Rush test before every big event. Seat logic is checked by the database itself.
Too much work for one person	Thin phases. Each phase is useful alone. The AI is last and can wait.

14. Where I learn from

Topic	Resource
Project I studied	CampusConnect: github.com/Srivastava-Shrestha/CampusConnect
FastAPI basics	fastapi.tiangolo.com (Tutorial and Advanced User Guide)
Full app example	github.com/fastapi/full-stack-fastapi-template
Next.js	nextjs.org/learn (free course)
UI parts	ui.shadcn.com and tailwindcss.com/docs
Database	postgresql.org/docs and docs.sqlalchemy.org (2.0 tutorial)
Google sign-in	developers.google.com/identity/gsi/web ("Verify the Google ID token")
Security	OWASP Cheat Sheet Series (cheatsheetseries.owasp.org) and OWASP ASVS
Badges standard	Open Badges 3.0 by 1EdTech (imglobal.org/spec/ob/v3p0)
Deploying	vercel.com/docs , render.com/docs , neon.tech/docs , upstash.com/docs
Rush testing	grafana.com/docs/k6
Learning paths	roadmap.sh (frontend, backend, DevOps)
AI (Phase 6)	docs.anthropic.com (Claude API, tool use)

15. Word list

Word	Simple meaning
Frontend	The part you see and click in the browser.
Backend	The part on the server that follows the rules and keeps the data.
API	The list of doors the website uses to ask the server for things.
Database	Where all the data is kept, in tables.
Deploy	Putting a new version online for people to use.
Session / cookie	A small note in your browser that says "this person is signed in".
CSRF token	A secret word our site adds to each change, so other sites can't fake clicks.

Audit log	A diary of who changed what, and when.
CDN	Copies of pages saved close to users, so they load fast.
Pull request	A request to add a change to the main code, after checks.
CI (GitHub Actions)	A robot that runs all the tests on every change.
Preview link	A private test copy of the site for one change.